

Multi-AZ Resilience: ALB, Auto Scaling Groups, and Recovery-Time SLOs

Course: Cloud Computing **Estimated Time:** 150–180 minutes **Difficulty:** Advanced **Prerequisites:** Previous lab completed (S3 bucket, IAM roles/instance profiles, Launch Templates, local `lab03/` app code, `.env` pattern, key pair, and a working `create_app_stack.sh` / `delete_app_stack.sh` pair).

This version builds directly on the previous lab's `create_app_stack.sh` and `delete_app_stack.sh` — the same two files, with the ALB/ASG/multi-AZ layer appended inline. It does not introduce a `lib/` of shared functions; uploader and viewer logic stays written out in full, the same style as the previous lab.

1. Learning Objectives

By the end of this lab, you will be able to:

1. Deploy the Uploader and Viewer behind an **Application Load Balancer (ALB)**, spread across **two Availability Zones** (`us-east-2a` , `us-east-2b`), instead of as single standalone instances.
2. Use an **Auto Scaling Group (ASG)** — attached to a target group and referencing the **same Launch Template** from the previous lab — as the mechanism that automatically replaces a terminated instance.
3. Define a **recovery-time SLO** (Service Level Objective), terminate a running instance, and measure whether the system recovers within that budget.
4. Write a **Python tool** that drives the experiment: terminate an instance via the ASG API, poll the ALB endpoint, and report a pass/fail recovery time against the SLO.
5. **Extend** the `create_app_stack.sh` / `delete_app_stack.sh` scripts from the previous lab directly — adding the ALB/ASG/multi-AZ layer on top of the exact bucket/IAM/launch-template logic you already wrote, rather than starting over.
6. Tighten the app security group so instances only accept traffic from the ALB, not the open internet — since the ALB is now the only intended entry point.
7. Explain why this design stays within (or draws sensibly from) AWS Free Tier limits.

2. Why This Matters

So far, “the uploader is down” has meant SSHing in or staring at a dead browser tab. Production teams don’t work that way — they define a **Service Level Objective (SLO)**: a measurable target for how the system should behave (e.g., “95% of requests succeed,” “recovery from a failed instance takes under 3 minutes”), backed by a **Service Level Indicator (SLI)** — the actual metric you measure — and they build infrastructure that **self-heals** without a human racing to relaunch a server by hand.

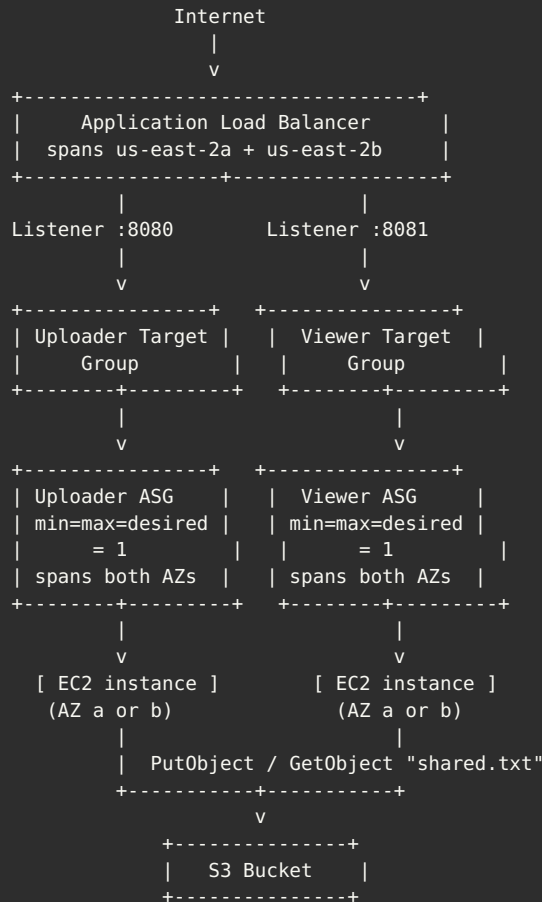
Today’s lab builds the smallest version of that pattern:

- An **ALB** is the single stable entry point — its DNS name doesn’t change even as the instances behind it come and go.
- A **Target Group** continuously health-checks the instances registered to it, and stops sending traffic to ones that fail.
- An **Auto Scaling Group** notices when its instance count drops below the desired count (because you terminated one, or it failed a health check) and launches a replacement **from the exact same Launch Template you already built** — no manual `run-instances` required.
- A **Python script** plays the role of an automated chaos/recovery test: kill something on purpose, measure how long the system takes to heal, and check that against your SLO — the same shape as a real incident postmortem’s “time to recovery” metric, just automated and repeatable.

⚠ **New Free Tier consideration: the ALB itself.** Unlike EC2/S3, an Application Load Balancer’s Free Tier

allowance is **750 hours + 15 LCUs per month, for 12 months, on legacy (pre-July 15, 2025) accounts.** On the newer credit-based accounts, ALB usage simply draws down your \$200 credit — there's no separate always-free ALB allowance. Either way, an ALB left running past its allowance costs roughly **\$0.0225/hour plus usage**, so **always run the delete script at the end of this lab.** This is in addition to — not instead of — watching your two EC2 instances' hours.

3. Architecture Overview



What's reused, unchanged, from the previous lab: the S3 bucket, both IAM roles and instance profiles, both Launch Templates (and the local `lab03/uploader-app` / `lab03/viewer-app` code they're built from), the key pair, and the AMI.

What's new: the ALB, its security group, two target groups, two Auto Scaling Groups, and a tightened app security group. Each Auto Scaling Group's desired capacity is **1** — this isn't about scaling *up*, it's about the ASG's other job: noticing a missing instance and replacing it. That's exactly what you'll trigger and measure.

4. Part A — Look Up Your VPC and Multi-AZ Subnets

The ALB needs subnets in at least two AZs. Your default VPC already has one — you're just looking up the IDs.

```
VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text --region us-east-2)
echo "$VPC_ID"

SUBNET_A=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=us-east-2a" \
  --query "Subnets[0].SubnetId" --output text --region us-east-2)

SUBNET_B=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=us-east-2b" \
  --query "Subnets[0].SubnetId" --output text --region us-east-2)

echo "us-east-2a subnet: $SUBNET_A"
echo "us-east-2b subnet: $SUBNET_B"
```

✓ **Checkpoint:** Both `$SUBNET_A` and `$SUBNET_B` print real subnet IDs (`subnet-...`), not `None`. If either is `None`, your default VPC may not have a subnet in that AZ — check `aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC_ID" --region us-east-2` to see what AZs you actually have, and use two of those instead of `us-east-2a` / `us-east-2b` throughout this lab.

These are read-only lookups — nothing to clean up here.

5. Part B — ALB Security Group and Tightening the App Security Group

Read through this part to understand the two security groups involved; you'll run the actual commands via the script in Part E, since they depend on resources ordering that's easiest to get right in the script.

The ALB security group is new, and is the only one that should be open to the whole internet — because the ALB is now the only intended entry point:

```
aws ec2 create-security-group \
  --group-name itmo-444-544-alb-sg \
  --description "Allow inbound to the ALB on the app listener ports" \
  --vpc-id "$VPC_ID" \
  --region us-east-2

aws ec2 authorize-security-group-ingress --group-id <alb-sg-id> --protocol tcp --port 8080 --cidr 0.0.0.0/0
region us-east-2
aws ec2 authorize-security-group-ingress --group-id <alb-sg-id> --protocol tcp --port 8081 --cidr 0.0.0.0/0
region us-east-2
```

The app security group (`itmo-444-544-app-sg`, from the previous lab) currently allows port 3000 from `0.0.0.0/0`. Now that the ALB fronts it, that's more open than it needs to be — instances should only accept traffic *from the ALB*, not directly from the internet:

```
aws ec2 revoke-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --cidr 0.0.0.0/0 --region us-east-2

aws ec2 authorize-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --source-group <alb-sg-id> --region us-east-2
```

`--source-group` instead of `--cidr` means “traffic is allowed only if it comes from something wearing this other security group” — not a specific IP range. This is why the ALB SG must exist *before* you can tighten the app SG: the rule directly references it.

✓ **Checkpoint:** You can explain, in one sentence, why `--source-group` is a tighter rule than `--cidr 0.0.0.0/0`, even though both currently let traffic on port 3000 through.

6. Part C — Target Groups, Load Balancer, and Listeners

Also a read-through — Part E's script runs the real, idempotent versions of these.

A **Target Group** is what the ALB actually health-checks and routes to — not the instance directly:

```
aws elbv2 create-target-group \
  --name itmo-444-544-uploader-tg \
  --protocol HTTP --port 3000 \
  --vpc-id "$VPC_ID" \
  --target-type instance \
  --health-check-protocol HTTP \
  --health-check-path / \
  --health-check-interval-seconds 10 \
  --health-check-timeout-seconds 5 \
  --healthy-threshold-count 2 \
  --unhealthy-threshold-count 2 \
  --region us-east-2
```

💡 The defaults for a target group's health check are a 30-second interval and a 5-check healthy/unhealthy threshold — safe for production, but that alone adds **2.5+ minutes** to any recovery measurement before the ALB even notices. This lab tightens those to a 10-second interval and a 2-check threshold, so your SLO experiment isn't dominated by health-check timing you didn't choose on purpose. Real teams make this same trade-off deliberately: faster detection costs a few more health-check requests per minute (negligible), and buys meaningfully faster failover.

The **Application Load Balancer** itself spans both subnets:

```
aws elbv2 create-load-balancer \
  --name itmo-444-544-app-alb \
  --subnets "$SUBNET_A" "$SUBNET_B" \
  --security-groups <alb-sg-id> \
  --scheme internet-facing \
  --type application \
  --region us-east-2
```

This takes a minute or two to become `active` — the script waits for it properly instead of guessing with `sleep`.

Each **Listener** is a port on the ALB that forwards to one target group. Rather than path-based routing (which would mean changing the apps' routes), this lab uses **two ports on one ALB** — simpler, and the apps don't change at all:

```
aws elbv2 create-listener \
  --load-balancer-arn <alb-arn> \
  --protocol HTTP --port 8080 \
  --default-actions Type=forward,TargetGroupArn=<uploader-tg-arn> \
  --region us-east-2
```

✓ **Checkpoint:** You can explain, in one sentence, the difference between a target group and a listener.

7. Part D — Auto Scaling Groups: The Recovery Mechanism

An Auto Scaling Group's job here isn't scaling up — it's **maintaining a desired count**. With `min=max=desired=1`, an ASG does exactly one thing: whenever the instance count drops below 1 (because you terminated it, or a health check failed it), it launches a replacement, **from the Launch Template you already built**, into one of the AZs you gave it.

```
aws autoscaling create-auto-scaling-group \
  --auto-scaling-group-name itmo-444-544-uploader-asg \
  --launch-template "LaunchTemplateName=itmo-444-544-uploader-lt,Version=\$Latest" \
  --min-size 1 --max-size 1 --desired-capacity 1 \
  --vpc-zone-identifier "$SUBNET_A,$SUBNET_B" \
  --target-group-arns <uploader-tg-arn> \
  --health-check-type ELB \
  --health-check-grace-period 120 \
  --region us-east-2
```

Two flags matter more than they look:

- `--health-check-type ELB` — by default, an ASG only checks whether the *EC2 instance itself* is running (EC2 status checks). `ELB` instead makes the ASG trust the target group's health check — the same one that's actually calling `GET /` on your app. An instance whose EC2 status is fine but whose Node process crashed would otherwise sit there marked "healthy" forever.
- `--health-check-grace-period 120` — a fresh instance needs time to run `dnf install`, `npm install`, and start the service before it's fair to judge it unhealthy. Without this, the ASG could decide a perfectly good but still-booting instance has failed and start churning it.

`Version='<code>$Latest'</code>` is the same launch-template mechanic from the previous lab — any launch template version you publish (say, after editing `app.js`) is what the ASG uses for the *next* replacement, automatically.

✓ **Checkpoint:** You can explain, in one sentence, why `--health-check-type ELB` matters more here than it would for a single standalone instance.

8. Part E — Extend Your Create and Delete Scripts

This lab builds directly on `create_app_stack.sh` and `delete_app_stack.sh` from the previous lab — the same two files, with new sections added. Nothing about the bucket, IAM roles, or launch-template logic changes; what changes is **how instances get launched**: instead of a direct `run-instances --launch-template` call, the script now creates an ALB, two target groups, and two Auto Scaling Groups, and lets the ASGs launch the instances from those same launch templates.

E1. `create_app_stack.sh` — What Stays, What Goes, What's Added

Everything through building and publishing the launch templates is **identical** to the previous lab — same bucket check, same two IAM roles, same `base64`-embedded user-data, same `ensure-or-version` launch template logic. The one thing that's removed is the block that launched instances directly:

```
# REMOVE this block — the Auto Scaling Groups will launch instances instead
UPLOADER_ID=$(aws ec2 run-instances \
  --launch-template "LaunchTemplateName=$UPLOADER_LT_NAME,Version=\<code>$Latest'</code>" \
  --count 1 \
  --region "$AWS_REGION" \
  --query "Instances[0].InstanceId" --output text)
# ... and the matching VIEWER_ID block, the wait, and the describe-instances table
```

Everything below is **new**, appended after the launch-template section:

```
echo "== Looking up the default VPC and subnets in both AZs =="
VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=isDefault,Values=true" \
  --query "Vpcs[0].VpcId" --output text --region "$AWS_REGION")

SUBNET_A=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_A" \
  --query "Subnets[0].SubnetId" --output text --region "$AWS_REGION")

SUBNET_B=$(aws ec2 describe-subnets \
  --filters "Name=vpc-id,Values=$VPC_ID" "Name=availability-zone,Values=$AZ_B" \
  --query "Subnets[0].SubnetId" --output text --region "$AWS_REGION")

echo "== ALB security group =="
ALB_SECURITY_GROUP_ID=$(aws ec2 describe-security-groups \
  --filters "Name=group-name,Values=$ALB_SG_NAME" "Name=vpc-id,Values=$VPC_ID" \
  --query "SecurityGroups[0].GroupId" --output text --region "$AWS_REGION" 2>/dev/null)

if [ "$ALB_SECURITY_GROUP_ID" = "None" ] || [ -z "$ALB_SECURITY_GROUP_ID" ]; then
  ALB_SECURITY_GROUP_ID=$(aws ec2 create-security-group \
    --group-name "$ALB_SG_NAME" \
    --description "Allow inbound to the ALB on the app listener ports" \
    --vpc-id "$VPC_ID" \
    --query "GroupId" --output text --region "$AWS_REGION")
  aws ec2 authorize-security-group-ingress --group-id "$ALB_SECURITY_GROUP_ID" \
```

```

        --protocol tcp --port "$UPLOADER_LISTENER_PORT" --cidr 0.0.0.0/0 --region "$AWS_REGION" >/dev/null
    aws ec2 authorize-security-group-ingress --group-id "$ALB_SECURITY_GROUP_ID" \
        --protocol tcp --port "$VIEWER_LISTENER_PORT" --cidr 0.0.0.0/0 --region "$AWS_REGION" >/dev/null
else
    echo "ALB security group already exists, skipping."
fi

echo "== Tightening the app security group to only accept traffic from the ALB =="
aws ec2 revoke-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
    --protocol tcp --port 3000 --cidr 0.0.0.0/0 --region "$AWS_REGION" >/dev/null 2>&1 || true
aws ec2 authorize-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
    --protocol tcp --port 3000 --source-group "$ALB_SECURITY_GROUP_ID" --region "$AWS_REGION" >/dev/null 2>&1 || true

echo "== Target groups =="
UPLOADER_TG_ARN=$(aws elbv2 describe-target-groups --names "$UPLOADER_TG_NAME" --region "$AWS_REGION" \
    --query "TargetGroups[0].TargetGroupArn" --output text 2>/dev/null)
if [ "$UPLOADER_TG_ARN" = "None" ] || [ -z "$UPLOADER_TG_ARN" ]; then
    UPLOADER_TG_ARN=$(aws elbv2 create-target-group \
        --name "$UPLOADER_TG_NAME" \
        --protocol HTTP --port 3000 \
        --vpc-id "$VPC_ID" \
        --target-type instance \
        --health-check-protocol HTTP \
        --health-check-path / \
        --health-check-interval-seconds 10 \
        --health-check-timeout-seconds 5 \
        --healthy-threshold-count 2 \
        --unhealthy-threshold-count 2 \
        --region "$AWS_REGION" \
        --query "TargetGroups[0].TargetGroupArn" --output text)
else
    echo "Target group $UPLOADER_TG_NAME already exists, skipping."
fi

VIEWER_TG_ARN=$(aws elbv2 describe-target-groups --names "$VIEWER_TG_NAME" --region "$AWS_REGION" \
    --query "TargetGroups[0].TargetGroupArn" --output text 2>/dev/null)
if [ "$VIEWER_TG_ARN" = "None" ] || [ -z "$VIEWER_TG_ARN" ]; then
    VIEWER_TG_ARN=$(aws elbv2 create-target-group \
        --name "$VIEWER_TG_NAME" \
        --protocol HTTP --port 3000 \
        --vpc-id "$VPC_ID" \
        --target-type instance \
        --health-check-protocol HTTP \
        --health-check-path / \
        --health-check-interval-seconds 10 \
        --health-check-timeout-seconds 5 \
        --healthy-threshold-count 2 \
        --unhealthy-threshold-count 2 \
        --region "$AWS_REGION" \
        --query "TargetGroups[0].TargetGroupArn" --output text)
else
    echo "Target group $VIEWER_TG_NAME already exists, skipping."
fi

echo "== Application Load Balancer =="
ALB_ARN=$(aws elbv2 describe-load-balancers --names "$ALB_NAME" --region "$AWS_REGION" \
    --query "LoadBalancers[0].LoadBalancerArn" --output text 2>/dev/null)
if [ "$ALB_ARN" = "None" ] || [ -z "$ALB_ARN" ]; then
    ALB_ARN=$(aws elbv2 create-load-balancer \
        --name "$ALB_NAME" \
        --subnets "$SUBNET_A" "$SUBNET_B" \
        --security-groups "$ALB_SECURITY_GROUP_ID" \
        --scheme internet-facing \
        --type application \
        --region "$AWS_REGION" \
        --query "LoadBalancers[0].LoadBalancerArn" --output text)
    echo "Waiting for the ALB to become active..."
    aws elbv2 wait load-balancer-available --load-balancer-arns "$ALB_ARN" --region "$AWS_REGION"
else
    echo "Load balancer $ALB_NAME already exists, skipping."
fi

```

```

echo "== Listeners =="
EXISTING_PORTS=$(aws elbv2 describe-listeners --load-balancer-arn "$ALB_ARN" --region "$AWS_REGION" \
--query "Listeners[].Port" --output text 2>/dev/null)

if echo "$EXISTING_PORTS" | tr '\t' '\n' | grep -qx "$UPLOADER_LISTENER_PORT"; then
    echo "Listener on port $UPLOADER_LISTENER_PORT already exists, skipping."
else
    aws elbv2 create-listener \
        --load-balancer-arn "$ALB_ARN" \
        --protocol HTTP --port "$UPLOADER_LISTENER_PORT" \
        --default-actions "Type=forward,TargetGroupArn=$UPLOADER_TG_ARN" \
        --region "$AWS_REGION" >/dev/null
fi

if echo "$EXISTING_PORTS" | tr '\t' '\n' | grep -qx "$VIEWER_LISTENER_PORT"; then
    echo "Listener on port $VIEWER_LISTENER_PORT already exists, skipping."
else
    aws elbv2 create-listener \
        --load-balancer-arn "$ALB_ARN" \
        --protocol HTTP --port "$VIEWER_LISTENER_PORT" \
        --default-actions "Type=forward,TargetGroupArn=$VIEWER_TG_ARN" \
        --region "$AWS_REGION" >/dev/null
fi

ALB_DNS_NAME=$(aws elbv2 describe-load-balancers --load-balancer-arns "$ALB_ARN" \
--query "LoadBalancers[0].DNSName" --output text --region "$AWS_REGION")

echo "== Auto Scaling Groups (these launch the instances now, not run-instances) =="
EXISTING_UPLOADER_ASG=$(aws autoscaling describe-auto-scaling-groups \
--auto-scaling-group-names "$UPLOADER_ASG_NAME" --region "$AWS_REGION" \
--query "AutoScalingGroups[0].AutoScalingGroupName" --output text 2>/dev/null)
if [ "$EXISTING_UPLOADER_ASG" = "None" ] || [ -z "$EXISTING_UPLOADER_ASG" ]; then
    aws autoscaling create-auto-scaling-group \
        --auto-scaling-group-name "$UPLOADER_ASG_NAME" \
        --launch-template "LaunchTemplateName=$UPLOADER_LT_NAME,Version=\$Latest" \
        --min-size 1 --max-size 1 --desired-capacity 1 \
        --vpc-zone-identifier "$SUBNET_A,$SUBNET_B" \
        --target-group-arns "$UPLOADER_TG_ARN" \
        --health-check-type ELB \
        --health-check-grace-period 120 \
        --tags "ResourceId=$UPLOADER_ASG_NAME,ResourceType=auto-scaling-group,Key=Name,Value=uploader-
app,PropagateAtLaunch=true" \
        --region "$AWS_REGION"
else
    echo "Auto Scaling Group $UPLOADER_ASG_NAME already exists, skipping."
fi

EXISTING_VIEWER_ASG=$(aws autoscaling describe-auto-scaling-groups \
--auto-scaling-group-names "$VIEWER_ASG_NAME" --region "$AWS_REGION" \
--query "AutoScalingGroups[0].AutoScalingGroupName" --output text 2>/dev/null)
if [ "$EXISTING_VIEWER_ASG" = "None" ] || [ -z "$EXISTING_VIEWER_ASG" ]; then
    aws autoscaling create-auto-scaling-group \
        --auto-scaling-group-name "$VIEWER_ASG_NAME" \
        --launch-template "LaunchTemplateName=$VIEWER_LT_NAME,Version=\$Latest" \
        --min-size 1 --max-size 1 --desired-capacity 1 \
        --vpc-zone-identifier "$SUBNET_A,$SUBNET_B" \
        --target-group-arns "$VIEWER_TG_ARN" \
        --health-check-type ELB \
        --health-check-grace-period 120 \
        --tags "ResourceId=$VIEWER_ASG_NAME,ResourceType=auto-scaling-group,Key=Name,Value=viewer-
app,PropagateAtLaunch=true" \
        --region "$AWS_REGION"
else
    echo "Auto Scaling Group $VIEWER_ASG_NAME already exists, skipping."
fi

cat > stack_outputs.env << EOF
VPC_ID=$VPC_ID
SUBNET_A=$SUBNET_A
SUBNET_B=$SUBNET_B
ALB_SECURITY_GROUP_ID=$ALB_SECURITY_GROUP_ID
ALB_ARN=$ALB_ARN
ALB_DNS_NAME=$ALB_DNS_NAME

```

```
UPLOADER_TG_ARN=$UPLOADER_TG_ARN
VIEWER_TG_ARN=$VIEWER_TG_ARN
EOF

echo "Done. ALB DNS name: $ALB_DNS_NAME"
echo "Uploader: http://$ALB_DNS_NAME:$UPLOADER_LISTENER_PORT/"
echo "Viewer: http://$ALB_DNS_NAME:$VIEWER_LISTENER_PORT/"
echo "Give the target groups 1-2 minutes to report healthy – check with:"
echo "  aws elbv2 describe-target-health --target-group-arn $UPLOADER_TG_ARN --region $AWS_REGION"
```

```
chmod +x create_app_stack.sh
```

💡 `stack_outputs.env` holds resource identifiers this new section *generates* (the ALB's ARN and DNS name, target group ARNs, subnet IDs) — kept separate from `.env`, which still holds the config values *you* chose by hand. The Python tool in Part F reads both files. Add `stack_outputs.env` to `.gitignore` alongside `.env`.

E2. `delete_app_stack.sh` — What's Added

The previous lab's teardown (IAM roles, S3 bucket) still runs — it just now runs **after** a new block that tears down the ALB/ASG layer first, since the instances (and the security groups they depend on) need to be gone before the rest can clean up safely:

```

#!/bin/bash
set -e
export $(grep -v '^#' .env | xargs)
export $(grep -v '^#' stack_outputs.env | xargs)

# --- New: tear down the ALB/ASG/multi-AZ layer first ---

echo "=== Deleting Auto Scaling Groups (this also terminates their instances) ==="
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name "$UPLOADER_ASG_NAME" --force-delete --region "$AWS_REGION" 2>/dev/null || true
aws autoscaling delete-auto-scaling-group \
  --auto-scaling-group-name "$VIEWER_ASG_NAME" --force-delete --region "$AWS_REGION" 2>/dev/null || true

echo "Waiting for both Auto Scaling Groups to be gone..."
for ASG_NAME in "$UPLOADER_ASG_NAME" "$VIEWER_ASG_NAME"; do
  for i in $(seq 1 20); do
    EXISTING=$(aws autoscaling describe-auto-scaling-groups \
      --auto-scaling-group-names "$ASG_NAME" --region "$AWS_REGION" \
      --query "AutoScalingGroups[0].AutoScalingGroupName" --output text 2>/dev/null)
    if [ "$EXISTING" = "None" ] || [ -z "$EXISTING" ]; then
      echo "$ASG_NAME deleted."
      break
    fi
    sleep 15
  done
done

echo "=== Deleting the load balancer ==="
aws elbv2 delete-load-balancer --load-balancer-arn "$ALB_ARN" --region "$AWS_REGION" 2>/dev/null || true
aws elbv2 wait load-balancers-deleted --load-balancer-arns "$ALB_ARN" --region "$AWS_REGION" 2>/dev/null || true

echo "=== Deleting target groups ==="
aws elbv2 delete-target-group --target-group-arn "$UPLOADER_TG_ARN" --region "$AWS_REGION" 2>/dev/null || true
aws elbv2 delete-target-group --target-group-arn "$VIEWER_TG_ARN" --region "$AWS_REGION" 2>/dev/null || true

echo "=== Restoring the app security group to its previous-lab state ==="
aws ec2 revoke-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --source-group "$ALB_SECURITY_GROUP_ID" --region "$AWS_REGION" >/dev/null 2>/dev/null || true

aws ec2 authorize-security-group-ingress --group-id "$APP_SECURITY_GROUP_ID" \
  --protocol tcp --port 3000 --cidr 0.0.0.0/0 --region "$AWS_REGION" >/dev/null 2>/dev/null || true

echo "=== Deleting the ALB security group ==="
aws ec2 delete-security-group --group-id "$ALB_SECURITY_GROUP_ID" --region "$AWS_REGION" 2>/dev/null || true

rm -f stack_outputs.env uploader-lt-data.json viewer-lt-data.json user-data-uploader.sh user-data-viewer
uploader-policy.json viewer-policy.json trust-policy.json

# --- Unchanged from the previous lab: bucket + IAM teardown ---

echo "=== Removing IAM instance profiles and roles ==="
aws iam remove-role-from-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" \
  --role-name "$UPLOADER_ROLE_NAME" 2>/dev/null || true
aws iam delete-instance-profile --instance-profile-name "$UPLOADER_PROFILE_NAME" 2>/dev/null || true
aws iam delete-role-policy --role-name "$UPLOADER_ROLE_NAME" \
  --policy-name "${UPLOADER_ROLE_NAME}-policy" 2>/dev/null || true
aws iam delete-role --role-name "$UPLOADER_ROLE_NAME" 2>/dev/null || true

aws iam remove-role-from-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" \
  --role-name "$VIEWER_ROLE_NAME" 2>/dev/null || true
aws iam delete-instance-profile --instance-profile-name "$VIEWER_PROFILE_NAME" 2>/dev/null || true
aws iam delete-role-policy --role-name "$VIEWER_ROLE_NAME" \
  --policy-name "${VIEWER_ROLE_NAME}-policy" 2>/dev/null || true
aws iam delete-role --role-name "$VIEWER_ROLE_NAME" 2>/dev/null || true

echo "=== Emptying and deleting S3 bucket ==="
aws s3 rm "s3://$BUCKET_NAME" --recursive --region "$AWS_REGION" 2>/dev/null || true
aws s3api delete-bucket --bucket "$BUCKET_NAME" --region "$AWS_REGION" 2>/dev/null || true

echo "Cleanup complete."

```

```
chmod +x delete_app_stack.sh
```

⚠ Running the full script now tears down **everything** — ALB, ASGs, target groups, security groups, IAM roles, and the S3 bucket. If you want to keep the bucket/IAM/launch-template foundation around (say, to re-run this lab later without redoing Part B/C of the previous lab), comment out the last two sections before running it.

✓ **Checkpoint:** You can point to the exact line in `create_app_stack.sh` where the old `run-instances` call used to be, and explain what replaced it.

E3. Extend `.env`

Add these to your existing `.env` (everything from the previous lab stays as-is):

```
UPLOADER_TG_NAME=itmo-444-544-uploader-tg
VIEWER_TG_NAME=itmo-444-544-viewer-tg
UPLOADER_ASG_NAME=itmo-444-544-uploader-asg
VIEWER_ASG_NAME=itmo-444-544-viewer-asg
ALB_NAME=itmo-444-544-app-alb
ALB_SG_NAME=itmo-444-544-alb-sg
UPLOADER_LISTENER_PORT=8080
VIEWER_LISTENER_PORT=8081
AZ_A=us-east-2a
AZ_B=us-east-2b
RECOVERY_SLO_SECONDS=180
```

✓ **Checkpoint:** `./create_app_stack.sh` completes and prints two `http://<alb-dns-name>:port/` URLs that both load in a browser within a couple of minutes.

9. Part F — Python Recovery Testing Tool

Three small files, each with one job: one polls HTTP, one talks to the ASG, one orchestrates the experiment.

```
testing/
├─ requirements.txt
├─ http_probe.py      # polls a URL, reports up/down
├─ aws_helpers.py     # finds and terminates the live instance via the ASG API
└─ chaos_test.py      # orchestrates the experiment, prints PASS/FAIL
```

G1. `testing/requirements.txt`

```
boto3>=1.34
requests>=2.31
```

```
pip install -r testing/requirements.txt --break-system-packages
```

G2. `testing/http_probe.py`

```

"""Polls an HTTP endpoint and reports up/down transitions."""
import time
import requests

def probe_once(url: str, timeout: float = 3.0) -> bool:
    try:
        resp = requests.get(url, timeout=timeout)
        return resp.status_code == 200
    except requests.RequestException:
        return False

def wait_for_state(url: str, want_up: bool, poll_interval: float, max_wait: float, log_rows: list):
    """Poll `url` until it reaches the wanted state (up or down).

    Appends (elapsed_seconds, is_up) to log_rows on every poll. Returns the
    elapsed time in seconds when the target state was reached, or None if
    max_wait was exceeded.
    """
    start = time.monotonic()
    while True:
        elapsed = time.monotonic() - start
        if elapsed > max_wait:
            return None
        is_up = probe_once(url)
        log_rows.append((round(elapsed, 2), is_up))
        if is_up == want_up:
            return elapsed
        time.sleep(poll_interval)

```

G3. testing/aws_helpers.py

```

"""Talks to the Auto Scaling Group this lab manages."""
import boto3

def get_inservice_instance_id(asg_name: str, region: str):
    client = boto3.client("autoscaling", region_name=region)
    resp = client.describe_auto_scaling_groups(AutoScalingGroupNames=[asg_name])
    groups = resp.get("AutoScalingGroups", [])
    if not groups:
        raise RuntimeError(f"Auto Scaling Group '{asg_name}' not found in {region}.")
    for inst in groups[0].get("Instances", []):
        if inst.get("LifecycleState") == "InService":
            return inst["InstanceId"]
    return None

def terminate_instance_via_asg(instance_id: str, region: str) -> None:
    client = boto3.client("autoscaling", region_name=region)
    client.terminate_instance_in_auto_scaling_group(
        InstanceId=instance_id,
        ShouldDecrementDesiredCapacity=False,
    )

```

💡 `terminate_instance_via_asg` calls the **Auto Scaling** API, not `ec2.terminate_instances`.

`ShouldDecrementDesiredCapacity=False` tells the ASG “this instance is going away, but desired capacity is still 1 — replace it.” Terminating the instance directly through EC2 would still eventually trigger a replacement once the ASG notices, but going through the ASG API is the deliberate, documented way to simulate this and makes the experiment’s starting point unambiguous.

G4. testing/chaos_test.py

```

#!/usr/bin/env python3
"""
Terminates the running instance behind one tier's Auto Scaling Group and

```

measures how long the ALB takes to serve healthy traffic again, then checks that against a recovery-time SLO.

Usage:

```
python3 chaos_test.py --tier uploader
python3 chaos_test.py --tier viewer --slo-seconds 120
```

"""

```
import argparse
import csv
import sys
import time
```

```
from aws_helpers import get_inservice_instance_id, terminate_instance_via_asg
from http_probe import probe_once, wait_for_state
```

```
def load_env(path: str) -> dict:
```

```
    values = {}
    with open(path) as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, _, value = line.partition("=")
            values[key.strip()] = value.strip()
    return values
```

```
def main():
```

```
    parser = argparse.ArgumentParser(description="Measure recovery time after killing one instance.")
    parser.add_argument("--tier", choices=["uploader", "viewer"], required=True)
    parser.add_argument("--slo-seconds", type=float, default=None)
    parser.add_argument("--poll-interval", type=float, default=2.0)
    parser.add_argument("--max-wait", type=float, default=600.0)
    args = parser.parse_args()
```

```
    env = {**load_env(".env"), **load_env("stack_outputs.env")}
    region = env.get("AWS_REGION", "us-east-2")
    alb_dns = env.get("ALB_DNS_NAME")
    if not alb_dns:
        sys.exit("ALB_DNS_NAME not found – run create_app_stack.sh first.")
```

```
    slo_seconds = args.slo_seconds if args.slo_seconds is not None else float(env.get("RECOVERY_SLO_SECONDS", 120))
```

```
    if args.tier == "uploader":
        port = env.get("UPLOADER_LISTENER_PORT", "8080")
        asg_name = env.get("UPLOADER_ASG_NAME")
    else:
        port = env.get("VIEWER_LISTENER_PORT", "8081")
        asg_name = env.get("VIEWER_ASG_NAME")
```

```
    url = f"http://{alb_dns}:{port}/"
    print(f"Target: {args.tier} tier at {url}")
```

```
    print("Confirming the endpoint is healthy before we start...")
    if not probe_once(url):
        sys.exit("Endpoint is not responding now – fix that before running a chaos test against it.")
```

```
    instance_id = get_inservice_instance_id(asg_name, region)
    if not instance_id:
        sys.exit(f"No InService instance found in {asg_name}.")
    print(f"Terminating {instance_id} via the Auto Scaling Group...")
```

```
    log_rows = []
    t0 = time.monotonic()
    terminate_instance_via_asg(instance_id, region)
```

```
    print("Waiting for the endpoint to go down...")
    down_at = wait_for_state(url, want_up=False, poll_interval=args.poll_interval,
                             max_wait=args.max_wait, log_rows=log_rows)
```

```
    if down_at is None:
        print("Endpoint never went down within max-wait – the target group may have "
```

```

        "already had a healthy standby, or the termination didn't take effect.")

    print("Waiting for the endpoint to come back up...")
    recovered_at = wait_for_state(url, want_up=True, poll_interval=args.poll_interval,
                                  max_wait=args.max_wait, log_rows=log_rows)

    log_path = f"chaos_test_{args.tier}_{int(t0)}.csv"
    with open(log_path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["elapsed_seconds", "is_up"])
        writer.writerows(log_rows)
    print(f"Poll log written to {log_path}")

    if recovered_at is None:
        print(f"FAIL: endpoint did not recover within max-wait ({args.max_wait:.0f}s).")
        sys.exit(1)

    print(f"Recovery time: {recovered_at:.1f}s (SLO: {slo_seconds:.0f}s)")
    if recovered_at <= slo_seconds:
        print("PASS: recovery time met the SLO.")
        sys.exit(0)
    else:
        print("FAIL: recovery time exceeded the SLO.")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Run it from the `testing/` folder (or adjust the `.env / stack_outputs.env` paths if you run it elsewhere):

```

cd testing
python3 chaos_test.py --tier uploader
python3 chaos_test.py --tier viewer --slo-seconds 120

```

✓ **Checkpoint:** A run prints a `Recovery time: ...s` line, writes a `chaos_test_<tier>_<timestamp>.csv` poll log, and exits with a clear PASS or FAIL against the SLO.

10. Part G — Run the Experiment

1. `./create_app_stack.sh` and wait ~2 minutes for both target groups to report healthy (`aws elbv2 describe-target-health --target-group-arn <tg-arn> --region us-east-2`).
2. Confirm both apps work through the ALB: upload a `.txt` file at `http://<alb-dns>:8080/`, view it at `http://<alb-dns>:8081/`.
3. `cd testing && python3 chaos_test.py --tier uploader`. Watch it terminate the instance, then poll until recovery.
4. Open the CSV log and find the transition — the last `False` row before the run of `True` rows is roughly where the outage window ends.
5. Repeat for `--tier viewer`.
6. Try tightening `RECOVERY_SLO_SECONDS` (e.g., to 60) and re-run — see it FAIL, and discuss in your report what would need to change (faster health checks? a warm standby with `desired-capacity 2`? pre-baked AMIs instead of `npm install` at boot?) to hit a tighter SLO.
7. `./delete_app_stack.sh` when you're done, and confirm the ALB DNS name stops responding.

✓ **Checkpoint:** You have at least one PASS and one FAIL run's CSV log, and can explain in a sentence or two what changed between them.

11. Troubleshooting

Issue	Likely Cause	Fix
SUBNET_A or SUBNET_B prints None	Your default VPC has no subnet in that AZ	Use <code>aws ec2 describe-subnets --filters "Name=vpc-id,Values=\$VPC_ID" --region us-east-2</code> to find two AZs you actually have, and use those consistently
<code>create-load-balancer</code> fails: “At least two subnets in two different Availability Zones must be specified”	SUBNET_A / SUBNET_B resolved to subnets in the same AZ, or one is empty	Re-check Part A’s lookups before running the script
Target group shows targets as unhealthy for a long time	<code>--health-check-grace-period</code> hasn’t elapsed yet, or <code>npm install</code> is still running	Wait the full grace period (120s) before troubleshooting further; check <code>systemctl status uploader.service</code> over SSH if it’s been several minutes
<code>chaos_test.py</code> exits: “Endpoint is not responding now”	Target group isn’t healthy yet, or the ALB security group/listener isn’t set up	Confirm the URL loads in a browser first; re-run <code>create_app_stack.sh</code> (idempotent) if something’s missing
Recovery time is much longer than expected	Default (not tightened) health check settings, low <code>--health-check-grace-period</code> , or the instance is slow to <code>npm install</code>	Confirm the target group’s interval/threshold settings actually took (<code>aws elbv2 describe-target-groups</code>); consider whether a slow <code>npm install</code> — not detection time — is the bottleneck
<code>delete_app_stack.sh</code> hangs on ASG deletion	Instances failing to terminate cleanly, or a very slow region	Check <code>aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <name></code> manually; the script’s polling loop gives up after 5 minutes and tells you to check the console
<code>delete-security-group</code> fails for the ALB SG	ALB hasn’t fully finished deleting yet (its network interfaces still reference the SG)	The script already waits with <code>load-balancers-deleted</code> — if it still fails, wait another minute and re-run <code>delete_app_stack.sh</code>
Charges appear for the ALB on your billing dashboard	Left running past the Free Tier hours/LCUs (or past your credit balance, on newer accounts)	Always run <code>delete_app_stack.sh</code> promptly; an idle ALB still costs its hourly rate even with no traffic

12. Submission Checklist

- ☐ Screenshot of `create_app_stack.sh` completing, showing both `http://<alb-dns>:port/` URLs.
- ☐ Screenshot of both apps working through the ALB (uploader upload confirmation, viewer showing content).
- ☐ Screenshot of `aws elbv2 describe-target-health` showing both target groups healthy.
- ☐ The CSV poll log and console output from a `chaos_test.py` run that **passed** its SLO, for at least one tier.
- ☐ The CSV poll log and console output from a run that **failed** a tightened SLO.
- ☐ One paragraph explaining, using your own recovery-time numbers, what dominates the recovery window (health-check timing vs. `boot/npm install` time vs. something else).
- ☐ One paragraph explaining what changed in `create_app_stack.sh` / `delete_app_stack.sh` between the previous lab and this one — specifically, what replaced the old `run-instances` call and why.
- ☐ Screenshot of `delete_app_stack.sh` completing with no errors.

13. Key Terms Glossary

- **SLO (Service Level Objective)** – A measurable target for how a system should behave, e.g. “recover within 180 seconds.”
- **SLI (Service Level Indicator)** – The actual metric measured to check against an SLO — here, recovery time as measured by `chaos_test.py`.
- **TTR (Time to Recovery)** – How long a system takes to return to a healthy state after a failure; the specific SLI this lab measures.

- **Application Load Balancer (ALB)** – A Layer-7 load balancer that routes HTTP(S) traffic to target groups based on listeners and rules.
 - **Target Group** – The set of instances (or other targets) an ALB health-checks and routes to; the ALB never routes to an instance directly.
 - **Listener** – A port + protocol on a load balancer that forwards matching traffic to a specific target group.
 - **Auto Scaling Group (ASG)** – A resource that maintains a desired count of instances launched from a Launch Template, replacing any that disappear or fail health checks.
 - **Health check grace period** – The window after an instance launches during which an ASG won't act on a failed health check, giving it time to finish booting.
 - **LCU (Load Balancer Capacity Unit)** – The usage-based billing dimension for an ALB, on top of its hourly rate.
 - **Self-healing infrastructure** – A system designed to automatically detect and recover from certain failures without manual intervention.
-